

Challenge 2: Reconstruction and Optimization of a 256-bit Permutation

Crypto Contest Submission

19 July 2026

1 Summary

The recovered one-round permutation is

rotate $\rightarrow$ XOR $\rightarrow$ reverse all 32 bytes $\rightarrow$ add,

with word rotations

`rot = {43, 7, 29, 14}.`

The final implementation passes all 1,000 supplied one-round vectors, the supplied 20-round vector, and 100,000 deterministic randomized differential tests. The first optimization replaces the generic byte map with four 64-bit byte swaps. The final optimization composes rounds in pairs: the word reversal cancels after two rounds, exposing four independent scalar dependency chains. Ten pairs are fully unrolled, and a local BMI2 target lets GCC use non-destructive constant rotates under the supplied compile command. The same source has an adaptive score build: global BMI2 changes the core to `always_inline`, and a larger inline threshold integrates the public wrapper into the timing loop. Two repeated campaigns measured paired speedups of 1.116x and 1.118x over the default build.

2 Recovering the structure

Every component operation is invertible. Addition is inverted by subtraction modulo 2^{64} , XOR is self-inverse, and the map `shuffle_map[i] = 31-i` is also self-inverse. For every supplied input/output pair, I computed

```
t0 = output - constants1 // modulo 2^64 per word
t1 = reverse_all_32_bytes(t0)
t2 = t1 XOR constants2
```

and tested each permitted $r \in \{1, \dots, 63\}$ for

```
R0TL64(input[i], r) == t2[i].
```

The recovery script intersects the candidate sets from all 1,000 vectors rather than trusting one possibly degenerate word. Each word has one candidate: 43, 7, 29, and 14. Forward evaluation confirms the stated operation order on all supplied vectors.

3 Reference and first specialization

The direct reference calls the supplied operations in the recovered order:

```
rotate_words_left_64wise(state, rot);
xor_constants_256wise(state, constants2);
shuffle_bytes_256(state, shuffle_map);
add_constants_64wise(state, constants1);
```

The fixed map is a complete byte reversal, not an arbitrary permutation:

```
reverse32(x0,x1,x2,x3) =
    (BSWAP64(x3), BSWAP64(x2), BSWAP64(x1), BSWAP64(x0)).
```

Thus 32 indexed byte copies and two temporary arrays become four `__builtin_bswap64` operations. One specialized round is

```
y0 = BSWAP64(ROTL64(x3,14) ^ C2[3]) + C1[0]
y1 = BSWAP64(ROTL64(x2,29) ^ C2[2]) + C1[1]
y2 = BSWAP64(ROTL64(x1, 7) ^ C2[1]) + C1[2]
y3 = BSWAP64(ROTL64(x0,43) ^ C2[0]) + C1[3]
```

The constant shift counts permit immediate-count rotate instructions. The pre-optimization submission called a one-round, state-writing helper from the provided loop twenty times. A later register-resident twenty-round loop served as an experimental baseline.

4 Two-round composition

A round reverses the four word positions. Two reversals restore the original positions, so each two-round output depends only on the word at the same index. Let T_j denote the rotate, XOR, byte-swap, and destination addition associated with source word j . Two rounds are

```
x0 <- T3(T0(x0)) x1 <- T2(T1(x1))
x2 <- T1(T2(x2)) x3 <- T0(T3(x3)).
```

This form exposes four independent chains without round-by-round word moves. The helper expands this block ten times explicitly. GCC's local `target("bmi2")` attribute permits `RORX`, a non-destructive rotate that does not update condition flags, while the rest of the contest file still compiles with the supplied default command.

The helper attribute is conditional on `__BMI2__`. With the plain command it remains the aligned local-target `noinline` helper. With `-mbmi2 -finline-limit=2000`, caller and callee target options match, the core is forced into the public function, and GCC also inlines that function into `main`. This preserves a no-extra-flags compatibility path while enabling the faster path permitted by the statement's optimization-flag rule.

The statement permits external helper functions and restricts edits inside the provided 20-round loop. On its first iteration, that permitted line calls the helper for all twenty rounds and sets the supplied loop counter to 19. The helper applies the same permutation to any input state; it contains no benchmark-input or output hard-coding.

5 Candidate experiments

The experimental harness builds each candidate into a separate binary so that large unrolled functions do not alter one another's instruction-cache layout. Every candidate first passes the supplied vectors and 100,000 randomized differential cases. On an AMD EPYC 7B12 VM with GCC 12.2.0, `-O3 -march=native`, CPU 2, and 15 repeated samples, the screening result was:

Candidate	Median ns	MAD ns	vs. pre-opt.
Preserved pre-optimization form	41.894	2.834	1.000x
Register loop	40.655	3.083	1.031x
Paired AVX2 loop	39.718	0.131	1.055x
Paired scalar loop	37.751	2.649	1.110x
Paired AVX2 full unroll	39.417	0.086	1.063x
Paired scalar full unroll	36.824	2.617	1.138x
Paired scalar unroll + BMI2	36.249	3.195	1.156x
One-state AVX2	73.630	0.266	0.569x

The one-state AVX2 version is slower because each dependent round requires variable left and right shifts, OR, XOR, byte shuffle, half permutation, and addition. Four scalar words instead provide four shorter independent chains. A transposed four-state AVX2 version improves throughput, but the official API updates one state serially and cannot use that batching. The paired AVX2 loop was stable but slower than scalar BMI2 on this host. A non-BMI2 inline unroll was nearly tied with BMI2 in the contest-shaped test, so both remain useful target-machine controls.

5.1 Deep frontend and cache review

An additional 18-candidate sweep varied unroll factor, 16/32/64/128-byte alignment, literal constants, inlining, wrapper layout, SIMD, and chain scheduling. The submitted helper is 1,267 bytes, has no hot-body spills, and keeps four state words and eight constants in registers. Its four independent RORX/XOR/BSWAP/ADD chains run close to the simple 80-cycle dependency depth, so the remaining difference is mainly frontend and code layout.

A 715-byte half-unrolled loop measured 1.0867x in one paired session, but only 0.9712x in an immediate repeat and 0.9214x with `-march=native`. Alignment changes, embedded constants, and forced inline were neutral or slower; serialized chains reached only 0.7305x. Under the later cross-call inline flags, full/pair-loop/half-unroll medians were 37.279/38.618/38.727 ns. The full unroll therefore remains the arithmetic core.

5.2 Cross-call inlining

With only `-mbmi2`, GCC merges the private core into the public `permute_20rounds`, but `main`'s timing loop still calls that function. Adding `-finline-limit=2000` removes this second boundary. Assembly inspection found no core-external call, push/pop, or state/constant memory operand in the timed loop; all twelve values remain in registers across outer iterations. A default `-mbmi2`/full-inline ablation measured paired 1.000x/0.999x/1.124x, identifying the public-to-main boundary as the gain. GCC 12 generated byte-identical binaries with inline limits 700 and 2000. A later digest-pinned official GCC 13.3.0 build confirmed that the two complete binaries are also byte-identical in the target compiler. Explicit 32/64/128-byte alignment moved the inlined loop entry but measured paired 1.006x/0.991x/0.990x; all bootstrap intervals touched or crossed 1, so the default alignment remains. Earlier generic IRA/native controls also failed their confidence gates. Thus the recommended score command keeps neither extra flag without a real-255H confirmation.

The repository assembly audit locates the timed backedge between the final two `clock()` calls. Both inline limits produce the same normalized 1,216-byte, 322-instruction loop: 80 each of RORX, XOR, BSWAP, and ADD/LEA, with no call, push/pop, or state/constant memory operand. A portable forced-inline control reduced total text by 292 bytes and the loop to 1,060 bytes, but destructive ROL/ROR added a move and measured only 0.985x (95% CI 0.958–1.002x). Smaller code did not beat BMI2.

5.3 Second-wave alternatives

Byte swap commutes with XOR, so $B(u \oplus K) = B(u) \oplus B(K)$; moreover, XOR with 2^{63} equals addition of 2^{63} modulo 2^{64} . This allowed a pre-swapped constant and one-bit absorption, but did not remove an instruction. An apparent 1.095x–1.116x separate-process gain disappeared under same-process AB/BA timing and reversed function layouts (1.0146x and 0.9943x). Handwritten inline assembly measured 0.9975x.

Conjugating the opposite-phase transforms allowed two 2-lane SIMD chains and reduced the generated core from roughly 320 to 240 instructions. It still measured only 0.845x–0.848x because four independent scalar chains became two longer vector chains and each rotate required shift/shift/OR.

An exact 64-bit unary-function table needs 128 EiB per function, or 256 EiB for the two conjugacy classes. Empirical influence and mixed-derivative checks also rejected byte-independent XOR/add table decomposition: all 64 input/output-byte edges appeared, all 1,000 mixed XOR derivatives were nonzero, and a restricted 16-dimensional ANF reached degree 16 in 29 of 64 output bits. These observations do not prove a general circuit lower bound, but directly falsify the proposed small-table forms. Compiler scheduling, target clones, PGO/LTO, manual BMI2 scheduling, and cross-call batching likewise produced no portable valid winner; batching is incompatible with the serial state recurrence.

For the actual constants, exhaustive low-bit projections also reject moving XOR through ADD: the equation $(x \oplus C) + A = (x + B) \oplus E$ has no solution already at widths 5, 3, 10, and 4 for the four transforms. Folding bit 63 into ADD works for two constants but leaves a nonzero XOR mask. The four two-round linear bit permutations match neither one rotate nor one rotate/byte-swap composition. Exact alternatives measured 0.980x for post-BSWAP XOR, 0.983x for top-bit folding, 0.620x for arithmetic XOR, and 0.251x for 32-bit halves.

A final complete-binary screen compiled 120 GCC/link combinations and found 26 hot-loop streams. LTO, whole-program, semantic-interposition, and section-GC options left the hot loop byte-identical. An apparent IRA 1.0217x win vanished across four controlled offsets (0.985–0.995x, all intervals crossing one), and PGO/scheduling also lost. GCC 13.3’s `-mtune=alderlake` remains only a real-255H A/B candidate: an AMD codegen run gave 1.0148x, but neither that host nor the available approximate port model represents the judge’s heterogeneous cores.

5.4 Exact GCC 13.3 and fourth-wave search

The official `gcc:13.3.0` image was pinned by digest and used to compile the unmodified final source. The saved reproduction records the compiler, Binutils and source hashes, direct correctness results, and the exact timed-loop audit:

Build	.text	Timed loop	Audit/result
Default -O3	5,972 B	31 B / 6 insn	one helper call
BMI2 inline 700/2000	7,246 B	1,216 B / 322 insn	identical binaries
Alder tune	7,246 B	1,216 B / 322 insn	distinct schedule
Alder tune + IRA	7,199 B	1,210 B / 322 insn	third schedule

All inline loops have zero core-external calls, stack operations, and hot memory operands. All builds pass the supplied vectors and 100,000 direct random-state, random-constant one/twenty-round tests. GCC 13.3 emits the same complete binary for `-mtune=alderlake`, `raptorlake`, and `meteorlake`; it does not provide an `arrowlake` tune name.

A 34-configuration scheduler/allocator screen yielded 32 valid builds but only eight loop streams. LLVM-MCA 16’s approximate hybrid model predicts 125.06 cycles for generic, 123.62 for Alder, and 121.06 for Alder+IRA. On the available AMD host, a stricter 5,000,000-call, six-warm-up, 40-sample comparison gave Alder-to-Alder+IRA paired 1.011x (CI 1.001–1.015x) on CPU 0

and 1.008x (0.998–1.025x) on CPU 4. The wrong microarchitecture and inconsistent intervals prohibit promotion, but the third stream is a high-priority 255H candidate.

The constant-placement axis also closed without a winner. Moving XOR after BSWAP or before rotate is exact for arbitrary constants but retained all 320 core operations. Fresh schema-4, 24-sample campaigns on CPU 0/4 measured post-BSWAP at 0.959x/0.963x and pre-rotate at 0.965x/0.960x, with all four intervals below one. Forcing constants to memory created 160 hot memory operands and enlarged the loop to 2,015 bytes; two 21-sample campaigns measured 0.979x (CI 0.964–0.992x) and 0.988x (0.969–1.002x). Literal specialization violates the random-constant contract and still does not remove an operation. Deterministic checks show that the original, byte-swapped, inverse-rotated and top-bit-folded masks fit neither sign-extended `imm32` nor the only fully absorbable XOR masks $\{0, 2^{63}\}$.

5.5 Fifth-wave source and backend schedules

The four assignments in a two-round block are independent. A digest-pinned GCC 13.3 screen therefore compiled all 24 source orders under generic, Alder, and Alder+IRA profiles. Order 2, 1, 0, 3 reduced LLVM-MCA’s approximate generic/Alder result from 125.06/123.62 to 121.06 cycles; Alder+IRA remained 121.06. The generic/Alder loop shrank from 1,216 to 1,211 bytes while retaining 322 instructions and all 320 core operations. Its three profiles have no call, push/pop, spill, or hot memory operand and pass 100,000 random-state and random-constant tests at one and twenty rounds.

A separate 106-flag scheduler/layout screen produced nine loop streams; all 106 builds plus three source-order cross-products passed exact assembly audits. The nine shortlisted variants also passed the supplied vectors and the direct 100,000-case gate. With the approximate Alder model, `-fselective-scheduling2` and `-fno-schedule-insns2` gave 120.06 and 120.07 cycles versus 121.06 for Alder+IRA. Disabling the critical path heuristic gave 120.14; 64-byte alignment and LTO changed placement but not the 121.06-cycle loop stream. Combining source order with the first two flags did not stack the static gain. AMD diagnostics disagreed across CPUs and are not a proxy for Lion Cove or Skymont. These builds were added only to the 255H screen/confirm manifest; the incumbent remains unchanged.

The remaining target experiment is encoded as a guarded **probe-screen-confirm-decide** workflow. It pins a CPUID probe to every allowed logical CPU, combines core type with Linux topology signals, and leaves ambiguous LP-E classification provisional. A replacement requires two sessions, two physical representatives of each requested core type, matching compiler/source/-manifest hashes, direct correctness, and exact measured-binary SHA-256/assembly gates; reused artifact paths or hashes cannot count as another session. Each run binds a fresh 128-bit id into its index and benchmark JSON; canonical evidence and paired-sample hashes also reject whitespace-renamed copies. A canonical fingerprint additionally pins the autotuner and benchmark drivers, loop audit, independent oracle/verifier, official archive, Python, `objdump`, and `size` executables. A changed screen protocol or mismatched confirmation protocols force a fresh measurement, and a stale topology probe cannot start a new screen. Correctness metadata must explicitly record random states and random ADD/XOR constants at both one and twenty rounds; the runner parses the verifier’s exact count, seed, round, constant, and PASS output. The reference translation unit uses fixed neutral flags, while candidate flags apply only to its object and link. Source bytes are snapshotted once and both original and rewritten-performance hashes are retained. Nested topology records are validated through each cache-list element and malformed input fails closed without a traceback. An expanded balanced 15-case integration screen passed all 15 direct correctness checks and all 15 exact-binary audits; its 1,000-call timings are tool-regression evidence only. Two deliberately undersized AMD/GCC12 confirmation sessions kept the incumbent while naming every missing target/compiler/sample/warm-up/iteration/random-case gate. Every P-core campaign needs paired median ≥ 1.010 and an adjusted lower bound > 1.005 without E/LP-E regression. Multiple qualifying candidates deliberately keep the incumbent pending a

direct head-to-head.

6 Correctness

The following all pass:

- all 1,000 supplied one-round input/output pairs;
- the supplied 20-round pair;
- each candidate’s public functions compared with the generic reference after one and twenty rounds for 100,000 deterministic random states and random add/XOR constants; and
- separate complete binaries for the preserved previous and final `contest.c` files.

```
one_round_vectors=1000
twenty_round_vector=PASS
random_differential_cases=100000
selftest=PASS

one-round testvector verification: OK (1000 pairs checked)
20-round testvector verification: OK
```

7 Score-facing repeated benchmark

An earlier microbenchmark called a `NOINLINE` 20-round function on every outer iteration, whereas GCC completely inlined the pre-optimization submission into the supplied timing loop. This changes a measurement by several percent. Also, the ratio of two separately sorted medians differed by about four percent from the median of paired sample ratios in an observed run.

The benchmark compiles complete contest files separately and can assign flags per case, including two builds of the same source. It changes only the number of calls in the supplied timing loop, discards three whole-process warm-ups, pins one logical CPU, and uses cyclic/reversed case order. It retains all raw internal `clock()` and external wall samples, median, MAD, p05/p95, a deterministic bootstrap median interval, paired speedup, and outlier flags in JSON. Every process also reruns the official vector checks. Before timing, schema 4 compiles each `contest.c` with only `main` renamed, links it to an independent verifier translation unit, and directly calls that candidate. This keeps the oracle self-test distinct from candidate differential verification. For every case given an `-audit-mode`, it then audits the exact performance executable against that timed-loop contract before any warm-up or measured run. The recommended commands specify a mode for every case. The same JSON records the canonical measurement-protocol fingerprint so a later decision cannot silently mix runner, verifier, archive, Python, or binary-tool revisions. A candidate requiring verifier-only flag overrides is quarantined rather than promoted.

A 10,000,000-call, 15-sample campaign produced:

Contest-shaped implementation	Median	MAD	p05–p95
Previous scalar	36.799 ns	0.924 ns	35.766–45.773 ns
Two-round unroll + BMI2	34.669 ns	2.232 ns	32.384–46.657 ns

This historical pre-optimization/final-source comparison had median paired speedup 1.068x, MAD 0.045x, and a bootstrap 95% interval of 1.008x–1.105x. A separate, noisier 20,000,000-call campaign gave 1.019x and an interval of 0.991x–1.082x. Reporting both avoids selecting only the favorable shared-VM run.

The decisive experiment instead compiled the final source both with default flags and with `-mbmi2 -finline-limit=2000`. Two independent 3,000,000-call, 21-sample campaigns produced:

Affinity	Default	Inline	Paired	Bootstrap 95% CI
CPU 0	44.467 ns	39.859 ns	1.116x	1.110–1.143x
CPU 4	45.924 ns	40.801 ns	1.118x	1.095–1.140x

The paired MAD values were 0.0281x and 0.0322x. Exact GCC 13.3 inspection has already established inline-limit 700/2000 equality and call removal. The final Intel Core Ultra 7 255H experiment therefore compares the incumbent with Alder and Alder+IRA schedules, source order 2,1,0,3, and the two shortlisted backend scheduler streams in independent sessions pinned to recorded physical representatives of each core type; native tune remains diagnostic only.

8 Reproduction

From the repository root:

```
python3 solutions/benchmark_02_permutation.py \
--case default=submissions/02/contest.c \
--case inline=submissions/02/contest.c --baseline default \
--case-cflag inline=-mbmi2 \
--case-cflag inline=-finline-limit=2000 \
--audit-mode default=default-call-allowed \
--audit-mode inline=full-inline-320 \
--cpu auto --iterations 3000000 \
--warmups 3 --samples 21 --random-cases 100000 \
--json /tmp/challenge02-inline.json

python3 solutions/02_optimization/audit_inline_02.py \
--json /tmp/challenge02-assembly.json
python3 solutions/02_optimization/analyze_transform_lower_bound_02.py
python3 solutions/02_optimization/analyze_constant_placement_02.py \
--emit-dir /tmp/ch2-constants/candidates \
--json /tmp/ch2-constants/analysis.json
python3 solutions/02_optimization/reproduce_gcc133_02.py \
--json /tmp/challenge02-gcc133.json
python3 solutions/02_optimization/screen_gcc133_schedules_02.py \
--json /tmp/challenge02-gcc133-schedule-screen.json
python3 solutions/02_optimization/screen_gcc133_source_orders_02.py \
--json /tmp/challenge02-gcc133-source-orders.json
python3 solutions/02_optimization/screen_gcc133_layout_02.py \
--json /tmp/challenge02-gcc133-layout.json
python3 solutions/02_optimization/autotune_02_255h.py probe \
--compiler /path/to/gcc-13.3.0 \
--out /tmp/ch2-255h/topology.json
```

For the longer candidate matrix:

```
python3 solutions/02_optimization/run_benchmarks.py \
--profile native --iterations 7000000 \
--warmup-iterations 30000000 --repeats 21 \
--random-cases 100000
```

The 18-candidate frontend/cache sweep is reproducible with:

```
python3 solutions/02_optimization/run_deep_review_02.py \
--iterations 2000000 --warmup-iterations 300000 \
--samples 15 --random-cases 100000
```

The submitted source supports both the supplied compatibility command and the recommended score command:

```
gcc -O3 -Wall -Wextra -o contest contest.c
./contest

gcc -O3 -Wall -Wextra -mbmi2 -finline-limit=2000 \
-o contest contest.c
./contest
```

9 References

- ISO/IEC JTC1/SC22/WG14, [N1570: C11 committee draft](#): exact-width integers, unsigned modular arithmetic, and valid shifts.
- GCC, [Byte-Swapping Builtins](#): the fixed reverse-map specialization.
- GCC, [x86 Function Attributes](#): local BMI2 code generation under the supplied build command.
- GCC, [GCC 13.3 Optimize Options](#): inline thresholds, scheduling, IRA, PGO, and LTO experiments.
- LLVM, [llvm-mca Machine Code Analyzer](#): scheduling-model throughput and resource analysis, including the model-accuracy limitation of static estimates.
- Intel, [Intel 64 and IA-32 Optimization Reference Manual](#), and the [Intrinsics Guide](#): scalar/BMI2 and AVX2 candidate analysis.
- GCC, [GCC 13.3 x86 Options](#): Alder Lake tuning availability and why AMD `-march=native` data cannot stand in for the Intel target.
- Intel, [Core Ultra 7 255H specifications](#): 6 P, 8 E, and 2 LP-E cores and AVX2 as the listed vector extension.
- Intel, [Game Dev Guide for 12th Gen Intel Core Processor](#): hybrid CUID flag, per-logical-CPU leaf 0x1a core type, and affinity-pinned probing.
- Linux kernel, [CPU topology documentation](#) and the [CPU sysfs ABI](#): package/core/thread sibling and capacity cross-checks.
- Mytkowicz et al., [Producing Wrong Data Without Doing Anything Obviously Wrong!](#): code-layout and execution-order benchmark bias.
- Claude Carlet, [Boolean Functions for Cryptography and Coding Theory](#): ANF degree and finite-difference background for the table-separability checks.
- Google, [Benchmark User Guide](#): warm-up, repetitions, interleaving, statistics, and machine-readable output.
- NIST, [Measures of Scale](#): MAD as a robust dispersion report for noisy samples.